

SKILL LAB MILESTONE -1

A1. Identify issues And list it with reasons

The issues are ,

- 1.Redundancy – user,email,theatre and all are repeated everywhere
- 2.Updating issue – we cannot update a value easily because if we have to update email we have to update it everywhere
3. insertion issue – if we add something all will be put into same table which might be confusing
- 4.deleting issue – if we delete something other data might be effected to since all particular datas are not separated
- 5.multiple values – seat numbers might have multiple values which violates 1NF
- 6.2NF-user name depends on user and not user_id
- 7.3NF-theater location will depend on theatre name

A2.NORMALIZE THE SCHEMA

CREATE DATABASE entertainment;

CREATE TABLE user(

user_id INT **PRIMARY KEY**,

user_name **VARCHAR(100)**,

user_email **VARCHAR(100)**

);

INSERT INTO TABLE user(id,name,email)

VALUES

(1,"ananya","ananya@gmail.com"),

(2,"anu","anu@gmail.com"),

(3,"nidhi","nidhi@gmail.com");

CREATE TABLE movie(

movie_id **INT PRIMARY KEY**,

movie_name **VARCHAR(100),**

);

INSERT INTO TABLE movie(id,name)

VALUES

(1,"kantara"),

(2,"kantara 2"),

(3,"KGF");

CREATE TABLE actors(

actor_name **VARCHAR(100),**

actor_id **INT PRIMARY KEY,**

actor_gender **VARCHAR(100),**

actor_type **VARCHAR(100)**

);

CREATE TABLE movie_type(

movie_id **INT PRIMARY KEY,**

movie_type **VARCHAR(100),**

movie_language **VARCHAR(100),**

movie_time **VARCHAR(100)**

);

CREATE TABLE theater(

theater_name **VARCHAR(100),**

theater_id **INT PRIMARY KEY,**

theater_location **VARCHAR(100)**

);

```
CREATE TABLE seat(  
    seat_id INT PRIMARY KEY,  
    seat_name VARCHAR(100)  
);
```

```
CREATE TABLE show(  
    show_id INT PRIMARY KEY,  
    theater_id INT,  
    movie_id INT,  
    show_name VARCHAR(100),  
    show_time VARCHAR(100),  
    FOREIGN KEY(movie_id) REFERENCES movie(movie_id),  
    FOREIGN KEY(theater_id) REFERENCES theater(theater_id),  
  
);
```

```
CREATE TABLE bookings(  
    booking_id INT PRIMARY KEY,  
    user_id INT,  
    show_id INT,  
    total_price VARCHAR(100),  
    payment_status VARCHAR(100),  
    FOREIGN KEY(user_id) REFERENCES user(user_id),  
    FOREIGN KEY(show_id) REFERENCES show(show_id),  
  
);
```

```
CREATE TABLE booking_seat(  
    booking_seat_id INT PRIMARY KEY,  
    booking_id INT ,  
    seat_id INT,  
    FOREIGN KEY(booking_id) REFERENCES booking(booking_id),
```

FOREIGN KEY(seat_id) REFERENCES seat(seat_id)

);

1NF – for seat numbers there will be multiple inputs which will violate 1NF so creating a separate seat table

2NF- there is partial dependency here as user_name will depend on user and even in theatre and movie. It is removed by creating separate tables for those and linked to booking table

3NF-instead of saving everything in one table example bookings, genre,language,hero is stored in movie type and seat , category price in seat

A3. LIST ALL THE ASSUMPTIONS

1.each user is uniquely identified by user_id-assumes no two users share same identity

2.one booking is made by one user -a booking cannot belong to multiple users

3.a show represents a specific movie at specific interval and time-combines movie , theatre,show_time into one entity

4.a movie can have multiple shows in different theatre and time-theater location depends only on theatre

5.each theatre has a fixed location-theater_location depends only on theatre

6.each booking can include multiple seats-hence separate booking_seat table is required

7.each seat is uniquely identified by seat_id -avoids duplication of seat information

8.payment status is associated with each booking-payment is done per booking not seat

9.total price is calculated per booking-based on number of seats and pricing rules

10.user email is unique for each user

SECTION B- **DATABASE DESIGN FROM REQUIREMENTS**

MOVIE

1. Can a movie have multiple languages and genres
2. Should language and genre be stored as separate tables or simple attributes
3. Can the same movie run in multiple cities and theaters simultaneously

THEATER

1. can a theatre have multiple screens
2. does each screen have a fixed number of seats and layout
3. are seat types

SHOW-

1. can the same screen run multiple shows in a day
2. can two shows overlap on the same screen
3. is pricing fixed per show or dynamic based on seat type

SEAT

1. is seat availability tracked per show or screen
2. can seat types have different prices within the same show
3. can a seat be temporarily reserved before booking confirmation

Booking

1. can a user book multiple seats in one booking
2. can a booking include seats from different shows
3. can bookings be cancelled or modified

USER

Can a user have multiple bookings over time

Should user history be stored permanently

Is user email unique for each user

LOCATION

Can a theatre exist in only one city

Should cities and locations be stored as separate entities

B2.DATABASE DESIGN

USERS

1. User_id - PRIMARY KEY
2. User_name
3. User_email

MOVIE

1. movie_id - PRIMARY KEY
2. movie_name

MOVIE_DETAILS

1. movie_id-foreign key
2. language
3. genre

CITY

1. city_id-primary key
2. city_name

THEATRE

1. theatre_id-primary key
2. theatre_name
3. city_id-foreign key

SCREEN

1. screen_id-primary key
2. theatre_id-foreign key
3. screen_name

SEAT_TYPE

1. seat_type_id - primary key
2. type_name

SEAT

1. seat_id-primary key
2. screen_id-foreign key
3. seat_type_id-foreign key
4. seat_number

SHOW

1. show_id-primary key

2. movie_id-foreign key
3. screen_id-foreign key
4. show_time

SHOW_PRICING

1. show_id-foreign key
2. seat_type_id-foreign key
3. price

BOOKING

1. booking_id-primary key
2. user_id-foreign key
3. show_id-foreign key
4. booking_date
5. total_price
6. payment_status

BOOKING_SEAT

1. booking_id-foreign key
2. seat_id-foreign key

SHOW_SEAT

1. show_id-foreign key
2. seat_id-foreign key
3. status

B3.NORMALIZATION

BOOKING

1. booking_id-primary key
2. show_id
3. booking_date
4. total_price
5. payment_status

change:removed user_id

BOOKING_USER

1. booking_id-foreign key
2. user_id-foreign key

BOOKING_USER_SEAT

1. booking_id-foreign key
2. user_id-foreign key
3. seat_id-foreign key

SECTION C-**QUERIES AND TRANSACTIONS**

C1. RETRIEVE ALL BOOKINGS FOR A USER WITHIN A DATARANGE

SELECT

b.booking_id,

m.movie_name,

t.theatre_name,

s.show_time,

se.seat_number,

FROM Bookings b

JOIN Users u ON b.user_id = u.user_id

JOIN Shows s ON b.show_id = s.show_id

JOIN Movies m ON s.movie_id = s.movie_id

JOIN Screen sc ON s.screen_id = s.screen_id

JOIN Theatre t ON b.theatre_id = t.theatre_id

JOIN Booking_seat bs ON b.booking_id = bs.booking_id

JOIN Seat se ON bs.seat_id = se.seat_id

WHERE u.user_id=1

AND b.booking_date BETWEEN '2026-01-01' AND '2026-12-31';

C2.MOST FREQUENTLY BOOKED MOVIE

SELECT

m.movie_name,

COUNT(b.booking_id) AS total_bookings

```
FROM Bookingd b
JOIN Shows s ON b.show_id = s.show_id
JOIN Movie m ON s.movie_id = m.movie_id
GROUP BY m.movie_name
ORDER BY total_bookings DESC
LIMIT 1;
```

C3.SHOWS WITH TOTAL BOOKED AND AVAILABLE SEATS

```
SELECT
s.show_id,
m.movie_name,
s.show_time,
COUNT Shows s
JOIN Movie m ON s.movie_id = m.movie_id
JOIN Screen sc ON s.screen_id = sc.screen_id
JOIN Theatre t ON sc.theatre_id = t.theatre_id
JOIN Show_Seat ss ON s.show_id = ss.show_id
WHERE t.theatre_id =1
AND DATE(s.show_time) = '2026-04-28'
GROUP BY s.show_id, m.movie_name, s.show_time;
```

C4. TRANSACTION-BOOKING OPERATION

```
SELECT seat_id
FROM Show_Seat
WHERE show_id=101
AND seat_id IN (1,2,3)
AND status = 'AVAILABLE';
INSERT INTO Bookings(booking_id, user_id, show_id, booking_date,
total_price, payment_status)
```

```
VALUES(1001,1,101,NOW(), 500 .'Paid);  
INSERT INTO Booking_Seat(booking_id, seat_id)  
VALUES(1001,1),(1001,2),(1001,3);  
UPDATE Show_Seat  
SET status = 'Booked'  
WHERE show_id = 101  
AND seat_id IN (1,2,3);  
COMMIT;
```